3D Interactive Portfolio

Build plan and step-by-step prompts (Next.js + React Three Fiber)

What the reference site does

- Boot screen styled as a terminal (python train.py --model ...) with loss counter, step count, tok/s and a progress %.
- Low-poly 3D room: robot with a TV head at a desk, monitor reading PROJECTS, pegboard with lanyards and ID badges, speaker, bookshelf, bean bag with a cat, city at dusk through the window.
- Drag to look around. Pink diamond hotspots mark clickable objects.
- Floating bottom nav (Projects, Experience, About, Contact), name and title bottom-left, menu button top-right.
- Flat version fallback when WebGL is not available.

Section to object mapping: Projects = monitor, Experience = pegboard badges, About = robot or bookshelf, Contact = speaker or desk phone.

Stack

Need	Tool
Framework	Next.js (App Router) + TypeScript
3D	three, @react-three/fiber, @react-three/drei
Lighting and effects	@react-three/postprocessing (bloom, vignette, SSAO)
Animation	GSAP (camera moves, UI), Framer Motion (panels)
State	Zustand
Styling	Tailwind CSS
3D model	Blender to .glb, compressed with gltf-transform (Draco/Meshopt)
Audio	howler or native Web Audio API
Deploy	Vercel

Plan

Phase	Goal	Output
0	Project structure and global state	Folders, store, design tokens
1	Boot / loader screen	Terminal loader tied to real asset progress
2	3D canvas and camera rig	Drag-to-look with limits and easing
3	Build the room	Placeholder blocks first, then real model
4	Lighting and mood	Dusk lighting, neon strips, bloom
5	Hotspots and navigation	Diamonds, hover states, camera fly-to
6	Content panels	Projects, Experience, About, Contact UI
7	Flat fallback and mobile	A non-WebGL version
8	Audio and polish	Speaker toggle, idle animations

Phase	Goal	Output
9	Swap in real models, optimize	Compressed glb, 60fps target
10	SEO, accessibility, deploy	Lighthouse pass, OG image, live URL

Biggest decision: the 3D room itself. Model it in Blender, use a low-poly asset pack, or start with primitive boxes (Phase 3) and swap in a real model later. Recommended: boxes first, so the whole flow works before the art is finished.

How to use the prompts

- Paste the **Master Context** together with **Part 0** in your first message. The Master Context alone asks for no code.
- For each new chat session, paste the Master Context first, then the part you are on.
- Replace placeholders like [YOUR NAME], [MODEL-NAME], title and city with your details.
- Tell the assistant your Next.js version (npx next --version) and whether you use a src/ folder.
- Run one part at a time. Verify the checklist, then commit (for example: feat: part 2 camera rig).
- Paste errors back verbatim with the file name.

Master Context (paste first, every time)

```
I'm building a 3D interactive portfolio in Next.js (App Router, TypeScript, Tailwind).
The project is already created and linked to GitHub.

Concept: a low-poly 3D room at dusk. A robot with a TV head sits at a desk.
The site boots with a terminal-style loader, then the user drags to look around.
Pink diamond hotspots mark clickable objects. Clicking one flies the camera to it
and opens a content panel. A floating bottom nav has: 01 Projects, 02 Experience,
03 About, 04 Contact. Name + title sit bottom-left. A fallback flat version
exists for browsers without WebGL.

Stack: three, @react-three/fiber, @react-three/drei, @react-three/postprocessing,
GSAP, Framer Motion, Zustand, Tailwind.

Palette: bg #0f1220, deep navy, neon pink #ff7ab8, warm amber #ffb366, cyan accent.
Fonts: a bold condensed display font for the name, a monospace font for UI labels.

Rules: write complete files, not fragments. Tell me the file path for each file.
Keep components small. Use TypeScript types. Do not skip steps.
After each part, give me a checklist to verify it works.
```

Part 0: Foundation

Paste after the Master Context (new chat) or on its own (same chat).

```
Set up the project foundation:
1. Install dependencies (give me the exact npm command).
2. Create this folder structure: components/{canvas,ui,panels,loader},
   lib/, store/, data/, public/models/, public/audio/.
3. Create a Zustand store (store/useApp.ts) with: loadProgress, isLoaded,
   activeSection ('projects'|'experience'|'about'|'contact'|null),
   hoveredHotspot, cameraTarget, isMuted, webglSupported.
4. Create data/content.ts with typed placeholder data for projects, experience,
   about, and contact. I'll fill in the real content later.
5. Configure Tailwind with my palette and add two Google fonts via next/font.
6. Add a WebGL support check hook (lib/useWebGL.ts) that writes to the store.
Give me the verification checklist.
```

Part 1: Terminal loader

Paste after the Master Context (new chat) or on its own (same chat).

```
Build the boot loader (components/loader/BootScreen.tsx).
- Full-screen dark terminal look.
- Typed line: "$ python train.py --model [MODEL-NAME] --data ./room"
- Live fake-training readout: loss counter decreasing from ~2.6, "step N/6000",
  tok/s, "gpu N%", and a percentage bar.
- Tie progress to REAL asset loading using drei's useProgress.
- When loading hits 100%, hold ~600ms, then fade/slide out and set isLoaded.
- Include a small "Drag to look around" hint that appears after the loader.
- Respect prefers-reduced-motion.
Use Framer Motion for the exit.
```

Part 2: Canvas and camera rig

Paste after the Master Context (new chat) or on its own (same chat).

```
Create the 3D canvas shell.
1. components/canvas/Scene.tsx with an R3F <Canvas> (dpr [1,2], shadows,
   ACES tone mapping, Suspense).
2. components/canvas/CameraRig.tsx: drag-to-look using pointer movement, with
   clamped yaw/pitch (about +/-20 degrees), damping/lerp for smoothness, and a
   gentle idle sway. NOT free OrbitControls.
3. A fly-to function that reads cameraTarget from the store and animates
```

```
   position + lookAt with GSAP (1.2s, power3.inOut), and a way to return to
   the default view.
4. Put a test cube in the scene so I can confirm it works.
Make the canvas fixed and full-screen behind the UI.
```

Part 3: Build the room (blockout first)

Paste after the Master Context (new chat) or on its own (same chat).

```
Build the room using primitive meshes first, in components/canvas/Room.tsx
and sub-components, so I can swap in real models later.
Include: back wall and side wall with paneling, wooden herringbone floor, rug,
a large desk with a glowing edge strip, a monitor with "PROJECTS" text on it
(use drei <Text>), a window with a night-city skyline made of emissive boxes
and a few palm shapes, a wall pegboard, a wall shelf with books, three framed
posters, a bean bag, a cabinet with a speaker, and a desk lamp.
Add a robot with a TV-head made from boxes.
Make each object its own component with a clear name. Position them to match
this layout: window center-back, pegboard on the right wall, desk center,
shelf and posters upper-left, bean bag lower-left, speaker/cabinet right.
```

Part 4: Lighting and mood

Paste after the Master Context (new chat) or on its own (same chat).

```
Add lighting and post-processing in components/canvas/Lighting.tsx
and Effects.tsx.
- Warm dusk key light through the window, with soft shadows.
- Emissive neon strips (desk edge, shelf underside) in amber.
- Pink point light near the lamp, a cool fill light.
- Subtle fog.
- Bloom (selective, so only emissive things glow), vignette, and light
  noise/grain. Keep it cheap enough for ~60fps on a mid laptop.
- Slow animated light streaks from the window across the floor.
Expose a quality prop (low/high) that disables heavy effects.
```

Part 5: Hotspots and navigation

Paste after the Master Context (new chat) or on its own (same chat).

```
Build the interaction layer.
1. components/canvas/Hotspot.tsx: a floating pink diamond (billboarded, gentle
   bob and pulse) with hover scale + cursor pointer + an HTML label via drei <Html>.
2. Attach hotspots to: monitor -> projects, pegboard -> experience,
   robot -> about, speaker -> contact. Each has a camera position/lookAt target.
3. Clicking a hotspot or a nav item sets activeSection, flies the camera there,
   and opens that panel. Close/Esc returns the camera.
4. components/ui/BottomNav.tsx: floating glass pill with 01-04 items, active
   state, keyboard accessible.
5. components/ui/NameTag.tsx (bottom-left: name + pink dot + "TITLE - CITY")
   and a top-right menu button.
Sync hover between 3D and nav.
```

Part 6: Content panels

Paste after the Master Context (new chat) or on its own (same chat).

```
Build the four panels in components/panels/ using Framer Motion
(slide in from the side, glass/blur background, mono labels).
- ProjectsPanel: cards with title, description, tech tags, GitHub/live links,
  and a hover preview image.
- ExperiencePanel: timeline styled like ID badges/lanyards (company, role,
  dates, bullet achievements).
- AboutPanel: bio, skills grouped by category, downloadable resume button.
```

```
  - ContactPanel: email, LinkedIn, GitHub, and a working contact form (API route
    using Resend or Nodemailer) with validation.
All content comes from data/content.ts. Trap focus, close on Esc, and
make them scrollable on small screens.
```

Part 7: Flat fallback and mobile

Paste after the Master Context (new chat) or on its own (same chat).

```
Build the fallback and responsive behavior.
1. components/flat/FlatSite.tsx: a clean, fully designed 2D version with the
   same content, shown when WebGL is unavailable or the user clicks
   "Flat version" or the device is very low-powered.
2. Add a toggle between 3D and flat.
3. On mobile: touch-drag look-around, bigger hotspot hit areas, bottom nav
   that fits, panels as full-screen sheets, lower DPR and effects.
4. Add a GPU-tier check (drei <PerformanceMonitor> or detect-gpu) to
   auto-drop quality.
```

Part 8: Audio and polish

Paste after the Master Context (new chat) or on its own (same chat).

```
Add audio and finishing touches.
- Speaker hotspot toggles lo-fi ambient music (fade in/out), plus a mute button
  in the UI, defaulting to off (autoplay is blocked anyway).
- Subtle UI sounds on hover/click, optional.
- Idle animations: robot head bob and blink, cat breathing, lamp flicker,
  window lights twinkling, floating dust particles.
- Smooth page-load transition into the room.
- Custom cursor that reacts to hotspots.
```

Part 9: Swap in real models and optimize

Paste after the Master Context (new chat) or on its own (same chat).

```
Help me replace the blockout with real glTF models.
1. Show me how to export from Blender (or prepare a downloaded pack) and
   compress with gltf-transform (Draco/Meshopt + texture to WebP/KTX2).
2. Update Room.tsx to load /public/models/room.glb with useGLTF, preload it,
   and map named meshes to my hotspot components.
3. Bake lighting/AO into textures where possible.
4. Target: under 8MB total, 60fps on mid hardware, instanced repeated objects.
```

Part 10: SEO, accessibility and deploy

Paste after the Master Context (new chat) or on its own (same chat).

```
Finish the project:
- Metadata, Open Graph image, theme-color, sitemap, robots.
- Real text content in the DOM (hidden but readable) so SEO and screen readers
  work even though the scene is WebGL.
- Keyboard navigation, focus states, aria labels, reduced-motion mode.
- Lighthouse pass and fixes.
- Deploy to Vercel, connect the custom domain, and add analytics.
```

Tips for the flow

- **Order matters.** Do not start Part 5 until Part 2's camera works.
- **Commit after every part** so you can roll back.
- **Content comes last.** Use placeholder text until Part 6.

- **Model sources:** Sketchfab (check license), Poly Pizza, Kenney assets, or a Blender freelancer for the hero robot only.
- **Make it your own.** Use your own robot character, colors and layout, so the result is inspired by the original rather than a copy.